



CI/CD

Deploy Starbucks Clone Web App Clone with Jenkins CI/CD

AWS EC2 (Ubuntu 24.04)

Jenkins

Docker

GitHub

Node.js

MobaXterm

Akanksha Singh

Project Duration: 29 June 2026 – 1 July 2026

[linkedin.com/in/akanksha-singh-2780a6253](https://www.linkedin.com/in/akanksha-singh-2780a6253)

Original project documentation • Please do not repost without permission

Project Overview

This document walks through a complete, hands-on **CI/CD pipeline** built for a coffee-shop style web application (a front-end clone project). The goal was to take an application from a GitHub repository all the way to a running, publicly accessible container — automatically — using **Jenkins** for orchestration, **Docker** for packaging, and **AWS EC2** as the build and runtime server.

Beyond just listing steps, this write-up also documents every real error encountered along the way — from a missing shared library to Docker Hub authentication failures — along with the exact root cause and fix for each one. Troubleshooting real pipeline failures is where most of the learning in DevOps actually happens, so those sections are included in full.

Tech stack used

- | | | |
|------------------------------|------------------------------------|--------------------------|
| • AWS EC2 (Ubuntu 24.04 LTS) | • Jenkins (Pipeline / Jenkinsfile) | • Docker & Docker Hub |
| • GitHub (source control) | • Node.js / npm | • MobaXterm (SSH client) |

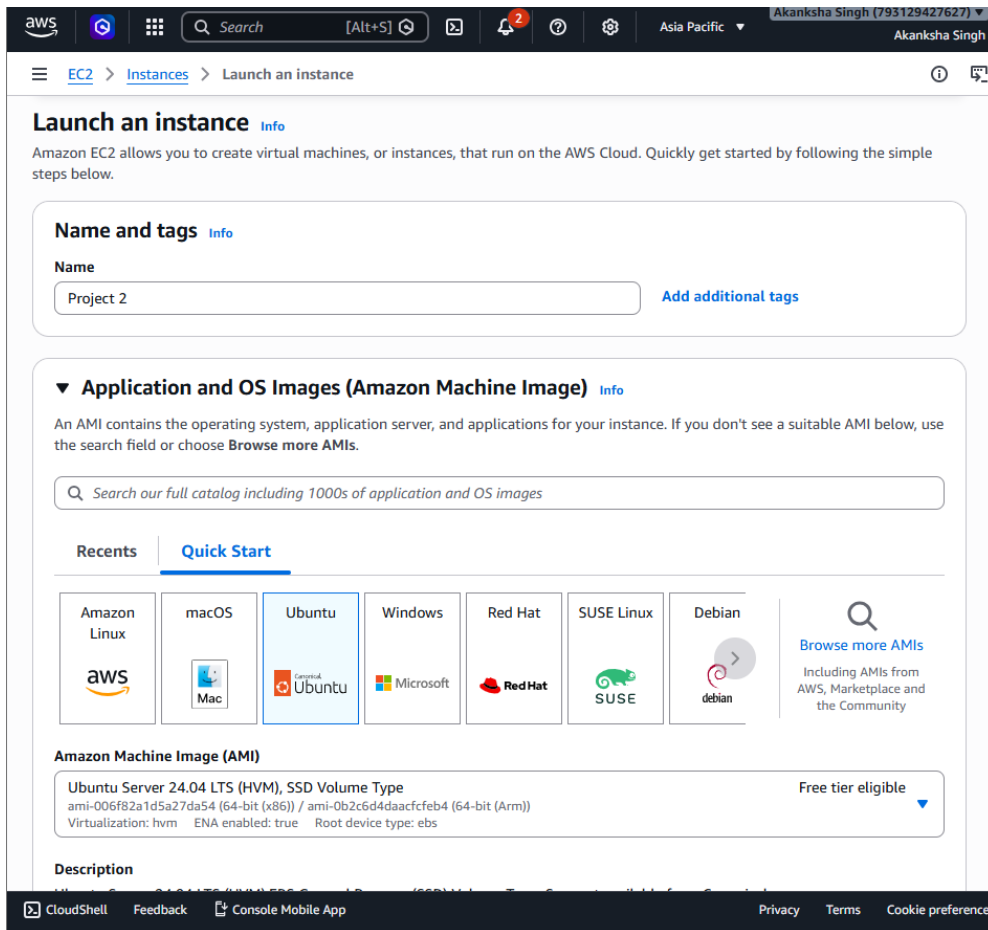
High-level pipeline flow

GitHub repository

- Jenkins **CI** job (checkout, install dependencies, build Docker image, push to Docker Hub)
- Jenkins **CD** job (pull the new image and run/refresh the container)
- live application served from the EC2 instance. The CD job is configured to trigger automatically whenever the CI job completes successfully, so a single push to GitHub can end with a freshly deployed container with no manual steps in between.

1 Provisioning the AWS EC2 Server

The first step is to provision a virtual server that will host both Jenkins (the automation engine) and Docker (the container runtime). An Ubuntu Server 24.04 LTS AMI was chosen because it is a stable, long-term-support release with excellent package support (apt) and is the most common base image used in production DevOps pipelines.



Launching a new EC2 instance from the AWS console with the Ubuntu 24.04 LTS AMI.

An **m7i-flex.large** instance type (2 vCPU / 8 GiB RAM) was selected instead of the default free-tier micro instance. Jenkins, Docker and a Node.js build process running together are memory-hungry, and an under-powered instance is one of the most common causes of pipelines hanging or getting silently killed by the kernel's out-of-memory manager. A key pair was also generated at this stage — this is the private key used later to SSH into the server securely instead of a password.

EC2 > Instances > Launch an instance

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
ami-006f82a1d5a27da54 (64-bit (x86)) / ami-0b2c6d4daacfcfeb4 (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs Free tier eligible

Description
Ubuntu Server 24.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).
Canonical, Ubuntu, 24.04, amd64 noble image

Architecture	AMI ID	Publish Date	Username
64-bit (x86)	ami-006f82a1d5a27da54	2026-06-10	ubuntu Verified provider

▼ Instance type Info | Get advice

Instance type
m7i-flex.large Free tier eligible
Family: m7i-flex 2 vCPU 8 GiB Memory Current generation: true
On-Demand SUSE base pricing: 0.15705 USD per Hour On-Demand RHEL base pricing: 0.12955 USD per Hour
On-Demand Ubuntu Pro base pricing: 0.10425 USD per Hour On-Demand Linux base pricing: 0.10075 USD per Hour
On-Demand Windows base pricing: 0.18815 USD per Hour
Additional costs apply for AMIs with pre-installed software

☐ All generations Compare instance types

▼ Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required
project Create new key pair

▼ Network settings Info Edit

Network Info
vpc-0f829601c384188a9

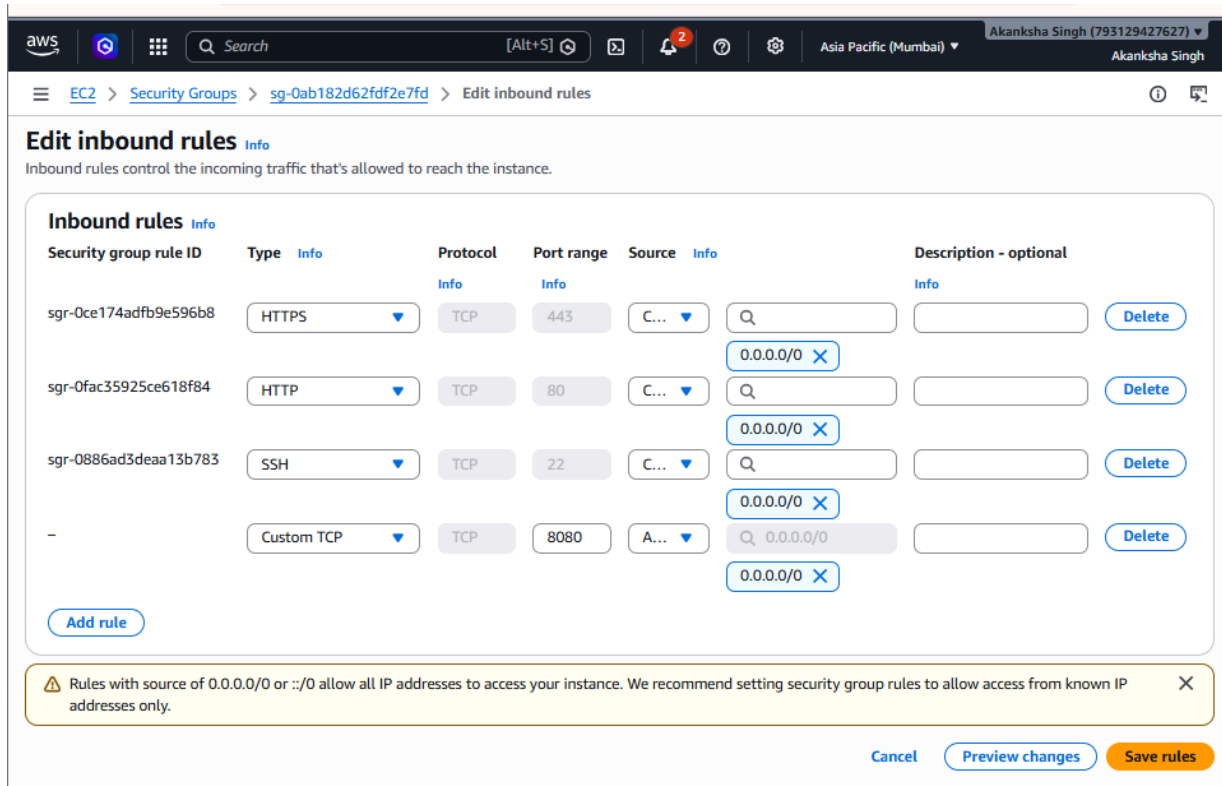
Subnet Info
No preference (Default subnet in any availability zone)

Auto-assign public IP Info

Instance type, key pair and network settings for the EC2 instance.

2 Configuring the Security Group (Firewall Rules)

A Security Group in AWS acts as a virtual firewall attached to the instance. By default, EC2 blocks all inbound traffic, so specific ports must be opened for the services this project needs to reach from the outside world.



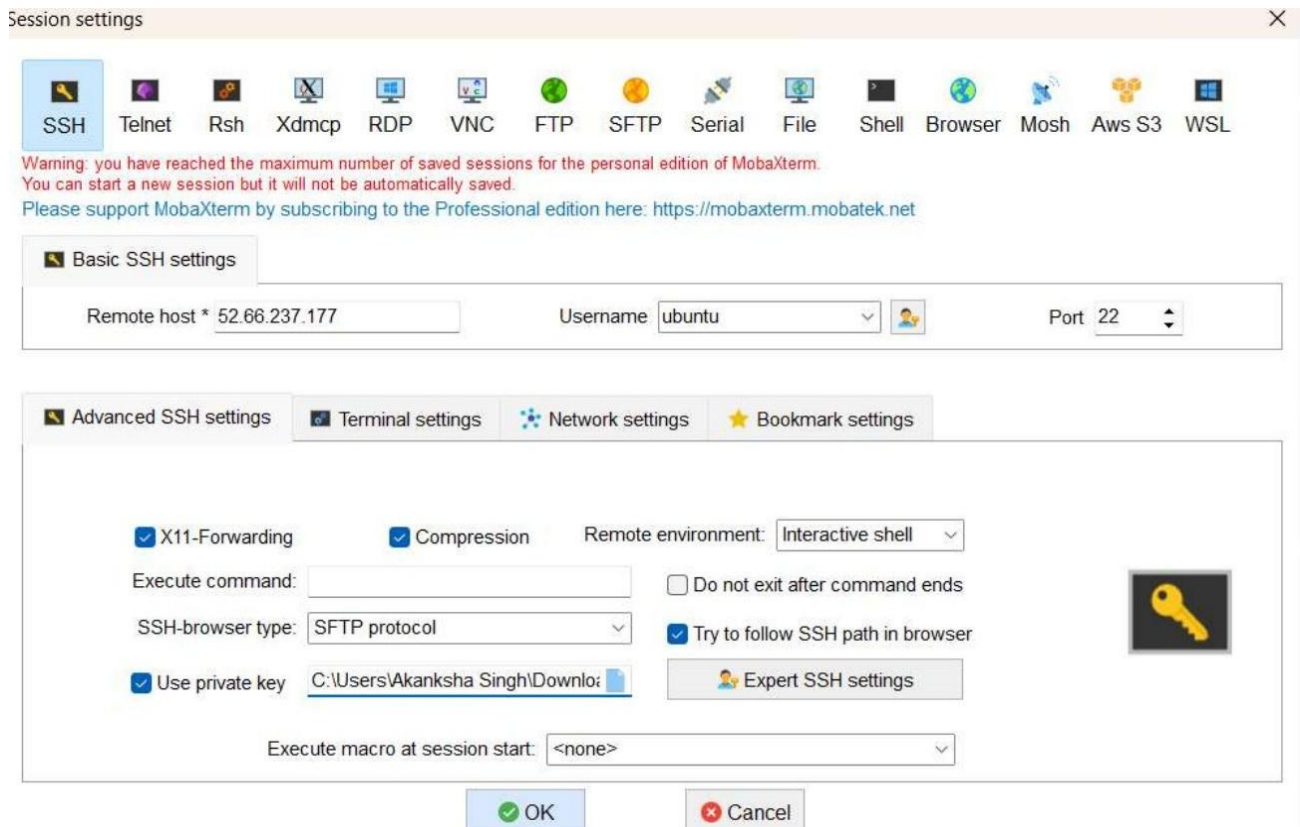
Inbound rules configured on the EC2 Security Group.

Command	Why we use it
Port 22 – SSH	Required to remotely log in to the server from a local machine (via MobaXterm/terminal) to run installation and configuration commands.
Port 80 / 443 – HTTP / HTTPS	Standard web traffic ports, opened for general web access and future TLS support.
Port 8080 – Custom TCP	The default port Jenkins listens on. Opening it allows the Jenkins web dashboard to be reached from a browser at <code>http://<public-ip>:8080</code> .
Port 3000 – Custom TCP (added later)	The port the containerized application listens on. This had to be opened separately once the app was deployed — covered in the troubleshooting section further below.

Note: the rules above use 0.0.0.0/0 as the source, meaning any IP address can reach these ports. This is fine for a personal learning project, but AWS explicitly warns against it for production systems — ideally, access should be restricted to known/trusted IP ranges.

3 Connecting to the Server via MobaXterm

With the instance running and the firewall configured, the next step is to open a secure shell (SSH) session into it. **MobaXterm** is a popular Windows SSH client with a built-in file browser, which makes it convenient for both running commands and transferring files.



MobaXterm session configured with the EC2 public IP, the 'ubuntu' user, and the downloaded private key.

The session uses the EC2 instance's public IP address, the default ubuntu login user for Ubuntu AMIs, and the private key file (.pem) that was downloaded when the key pair was created — this replaces the need for a password entirely and is significantly more secure.

4 Installing Jenkins

Jenkins is the automation server that will run our CI and CD jobs. It was installed with a single shell script executed over the SSH session, shown below with a line-by-line explanation.

```
#!/bin/bash

# Update system
sudo apt update -y

# Install Java 21 (Required for latest Jenkins)
sudo apt install -y fontconfig openjdk-21-jre

# Install required packages
sudo apt install -y curl wget gnupg

# Create keyrings directory
sudo mkdir -p /etc/apt/keyrings

# Download the latest Jenkins GPG key (2026)
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

# Add Jenkins repository
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable/binary/" | \
sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

# Update package list
sudo apt update -y

# Install Jenkins
sudo apt install -y jenkins

# Enable and start Jenkins
sudo systemctl enable jenkins
sudo systemctl start jenkins

# Check Jenkins status
sudo systemctl status jenkins --no-pager

# Display Jenkins initial admin password
echo ""
echo "====="
echo "Jenkins Initial Admin Password:"
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
echo ""
echo "====="
echo "Open Jenkins in your browser:"
echo "http://<YOUR\_PUBLIC\_IP>:8080"
echo "=====
```

Installation script executed on the Ubuntu server to set up Jenkins.

Command	Why we use it
<code>sudo apt update -y</code>	Refreshes the local package index so the server knows about the latest available package versions before installing anything.
<code>sudo apt install -y fontconfig openjdk-21-jre</code>	Jenkins runs on the Java Virtual Machine, so a Java runtime is a hard prerequisite. Java 21 is required for the current Jenkins release; fontconfig is needed for Jenkins' UI-related font rendering.
<code>sudo apt install -y curl wget gnupg</code>	Utility tools used to download files and verify package signatures in the next steps.
<code>sudo mkdir -p /etc/apt/keyrings</code>	Creates the directory where the Jenkins GPG signing key will be stored, following Debian/Ubuntu's modern (post-apt-key) convention for trusted repository keys.
<code>sudo wget -O ../jenkins-keyring.asc https://pkg.jenkins.io/...</code>	Downloads Jenkins' official GPG key, used to cryptographically verify that Jenkins packages actually come from the official Jenkins project and haven't been tampered with.
<code>echo "deb [signed-by=...] https://pkg.jenkins.io/debian-stable binary/" sudo tee /etc/apt/sources.list.d/jenkins.list</code>	Registers Jenkins' own package repository with apt, so 'apt install jenkins' knows where to fetch it from.
<code>sudo apt update -y</code>	Reloads the package index again, this time including the newly added Jenkins repository.
<code>sudo apt install -y jenkins</code>	Installs the Jenkins package itself.
<code>sudo systemctl enable jenkins && sudo systemctl start jenkins</code>	Enables Jenkins to auto-start on every server reboot, then starts the service immediately.
<code>sudo systemctl status jenkins --no-pager</code>	Confirms the service is active/running, and surfaces the process ID and recent log lines for a quick health check.
<code>sudo cat /var/lib/jenkins/secrets/initialAdminPassword</code>	Jenkins generates a one-time password on first install, written to this file. It's needed to unlock the web setup wizard in the next step


```

root@ip-172-31-3-198:~# history
 1 sudo apt update
 2 sudo apt install -y fontconfig openjdk-21-jre
 3 sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-
stable/jenkins.io-2026.key
 4 echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.i
o/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
 5 sudo apt update
 6 sudo apt install -y jenkins
 7 sudo systemctl enable jenkins
 8 sudo systemctl start jenkins
 9 sudo systemctl status jenkins
10 cat /var/lib/jenkins/secrets/initialAdminPassword
11 history
root@ip-172-31-3-198:~#

```

```

root@ip-172-31-3-198:~# sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-30 12:49:37 UTC; 59s ago
     Main PID: 5010 (java)
        Tasks: 46 (limit: 9269)
      Memory: 922.4M (peak: 923.0M)
         CPU: 14.536s
    CGroup: /system.slice/jenkins.service
            └─5010 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins>

Jun 30 12:49:32 ip-172-31-3-198 jenkins[5010]: [LF]> This may also be found at: /var/li>
Jun 30 12:49:32 ip-172-31-3-198 jenkins[5010]: [LF]>
Jun 30 12:49:32 ip-172-31-3-198 jenkins[5010]: [LF]> *****>
Jun 30 12:49:32 ip-172-31-3-198 jenkins[5010]: [LF]> *****>
Jun 30 12:49:32 ip-172-31-3-198 jenkins[5010]: [LF]> *****>
Jun 30 12:49:37 ip-172-31-3-198 jenkins[5010]: 2026-06-30 12:49:37.064+0000 [id=39] >
Jun 30 12:49:37 ip-172-31-3-198 jenkins[5010]: 2026-06-30 12:49:37.083+0000 [id=30] >
Jun 30 12:49:37 ip-172-31-3-198 systemd[1]: Started jenkins.service - Jenkins Continuou>
Jun 30 12:49:38 ip-172-31-3-198 jenkins[5010]: 2026-06-30 12:49:38.727+0000 [id=56] >
Jun 30 12:49:38 ip-172-31-3-198 jenkins[5010]: 2026-06-30 12:49:38.728+0000 [id=56] >

root@ip-172-31-3-198:~# cat /var/lib/jenkins/secrets/initialAdminPassword
5948582ddb9847e4bf43e27eaf2563fa

```

Confirming Jenkins is active (running) via systemctl, and retrieving the one-time initial admin password.

5 Unlocking Jenkins & Creating the Admin User

Navigating to `http://<public-ip>:8080` opens the Jenkins setup wizard. Pasting in the initial admin password proves that whoever is completing setup has shell access to the server, which is Jenkins' way of preventing a stranger from taking over a freshly installed instance.

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

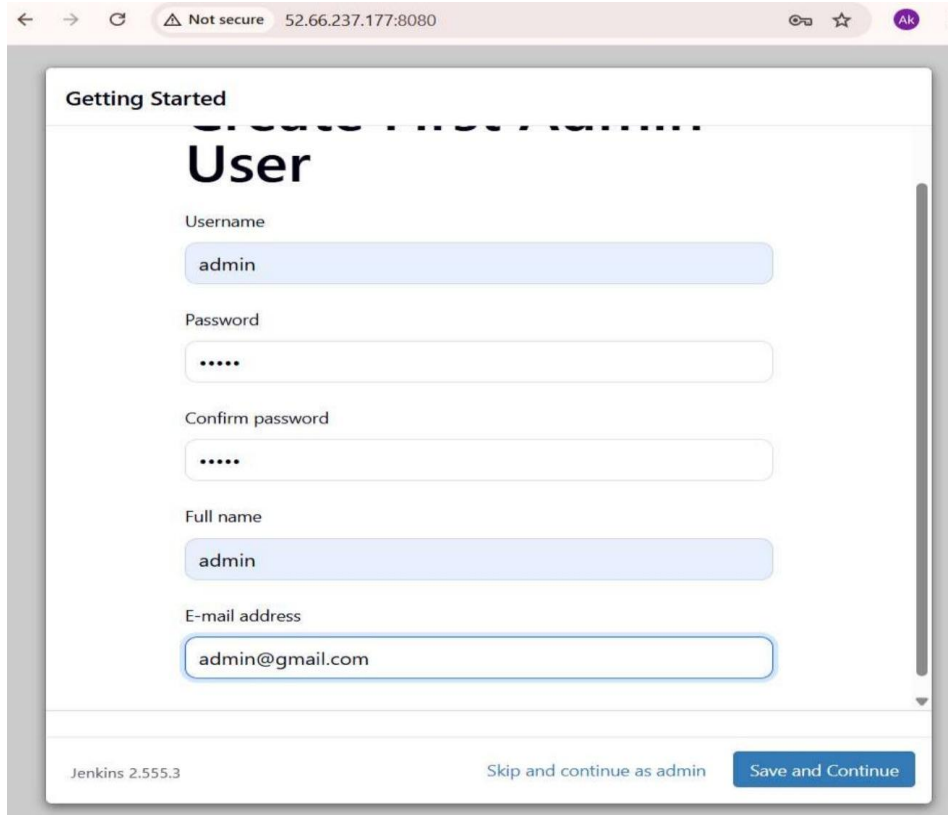
```
/var/lib/jenkins/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

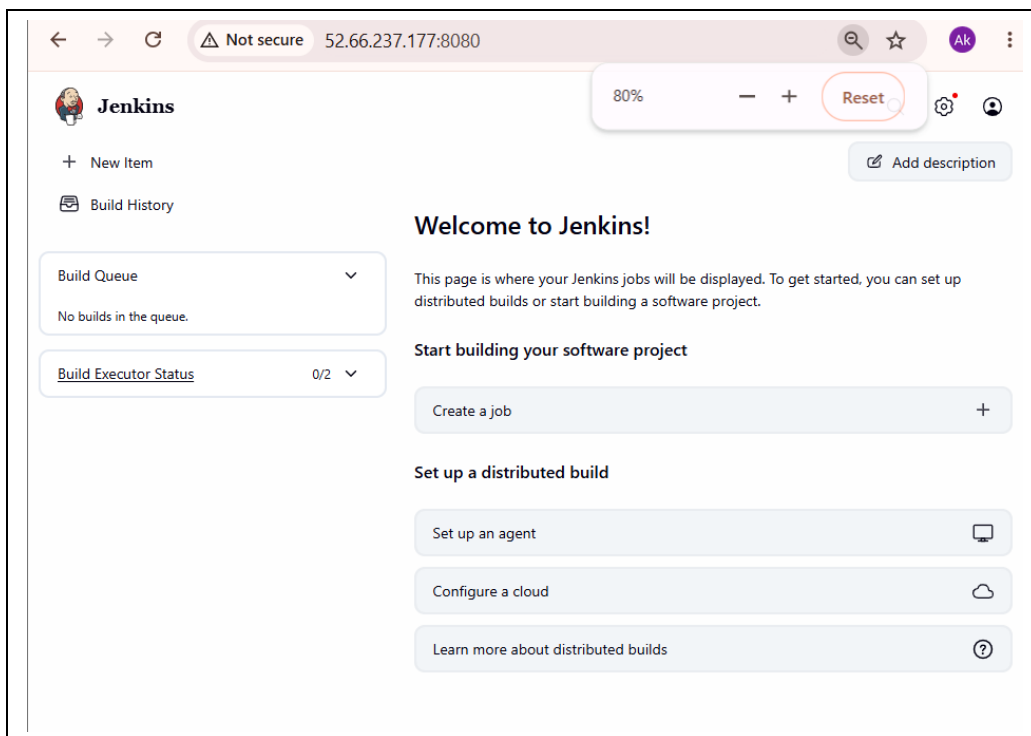
Continue

The 'Unlock Jenkins' screen, requesting the one-time password retrieved via SSH.



The screenshot shows the 'Getting Started' page of Jenkins. The form is titled 'User' and contains the following fields: Username (admin), Password (masked with dots), Confirm password (masked with dots), Full name (admin), and E-mail address (admin@gmail.com). At the bottom, there is a 'Skip and continue as admin' link and a 'Save and Continue' button. The Jenkins version 2.555.3 is displayed in the bottom left corner.

Creating the first admin account — this becomes the permanent login used for all further configuration.



The screenshot shows the Jenkins dashboard. The top navigation bar includes the Jenkins logo, a search icon, a star icon, and a user profile icon labeled 'Ak'. The main content area is titled 'Welcome to Jenkins!' and contains the following sections:

- Build Queue:** A dropdown menu showing 'No builds in the queue.'
- Build Executor Status:** A dropdown menu showing '0/2'.
- Start building your software project:** A button labeled 'Create a job' with a plus icon.
- Set up a distributed build:** Three buttons: 'Set up an agent' (with a computer icon), 'Configure a cloud' (with a cloud icon), and 'Learn more about distributed builds' (with a question mark icon).

6 Installing Docker on the Server

Docker is what packages the application into a portable, self-contained image and runs it as an isolated container. It's installed directly on the same EC2 instance as Jenkins, and Jenkins is later given permission to trigger Docker commands as part of the pipeline.

```

root@ip-172-31-3-198:~# # Add Docker's official GPG key:
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker
.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
# Add the repository to Apt sources:
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https:
//download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-c
ompose-plugin -y
sudo usermod -aG docker ubuntu
sudo chmod 777 /var/run/docker.sock
newgrp docker
sudo systemctl status docker
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Ign:4 https://pkg.jenkins.io/debian-stable binary/ InRelease
Hit:5 https://pkg.jenkins.io/debian-stable binary/ Release
Hit:6 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/sy
stemd/system/docker.service.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/system
d/system/docker.socket.
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-3-198:~#

```

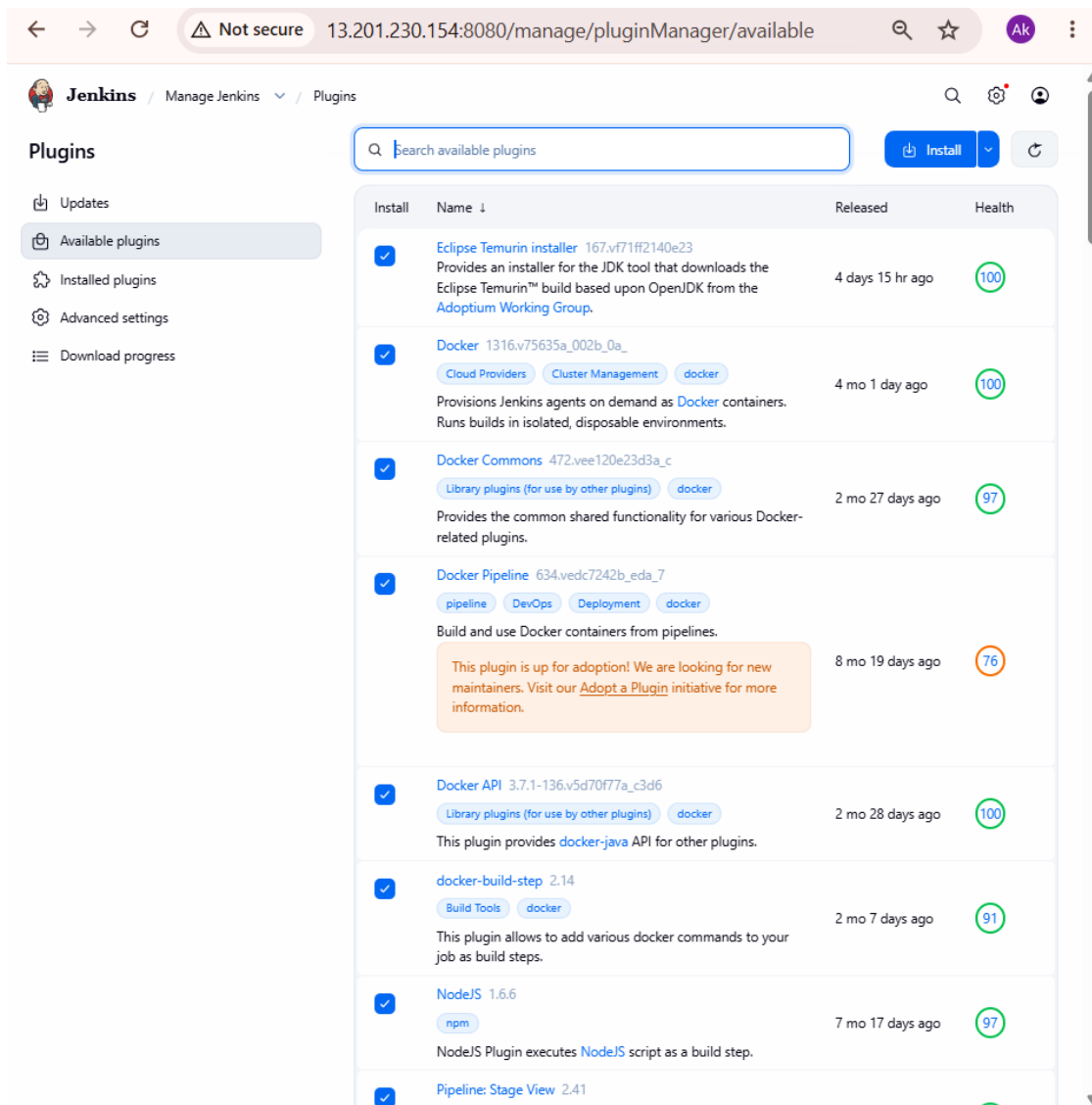
Adding Docker's official GPG key and repository, then installing the Docker engine and CLI tools.

```
# Add Docker's official GPG key:
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
# Add the repository to Apt sources:
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y
sudo usermod -aG docker ubuntu
sudo chmod 777 /var/run/docker.sock
newgrp docker
sudo systemctl status docker
```

Command	Why we use it
sudo install -m 0755 -d /etc/apt/keyrings	Prepares the keyrings directory (same pattern used for Jenkins) to store Docker's signing key.
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o ../docker.asc	Downloads Docker's official GPG key for package verification.
echo "deb [arch=\$(dpkg --print-architecture) signed-by=...] https://download.docker.com/linux/ubuntu ..."	Registers Docker's official apt repository, matched to the exact CPU architecture and Ubuntu release codename of this server.
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y	Installs the Docker engine, its CLI, the container runtime (containerd), and the buildx/compose plugins.
sudo usermod -aG docker ubuntu	Adds the 'ubuntu' user to the docker group so Docker commands can run without needing sudo every time.
sudo chmod 777 /var/run/docker.sock && newgrp docker	Relaxes permissions on the Docker socket so the Jenkins service user can also issue Docker commands — a common (though permissive) approach for single-user learning/demo environments.
sudo systemctl status docker	Confirms the Docker daemon is active and ready to build/run containers.

7 Installing Jenkins Plugins & Configuring Tools

Jenkins integrates with other tools in one of two ways: through a **plugin** (Jenkins manages and calls the tool internally) or by **installing the tool directly** on the server and simply pointing Jenkins at it. For this pipeline, both approaches were used — plugins for Docker and NodeJS integration, and direct tool auto-installers for the JDK, NodeJS and Docker versions used inside the pipeline itself.



Installing the required Jenkins plugins: Eclipse Temurin installer, Docker, Docker Commons, Docker Pipeline, Docker API and the NodeJS plugin.

After installing plugins, Jenkins was configured (under *Manage Jenkins* ▾ *Tools*) with three named tool installations that the Jenkinsfile references directly: a JDK 17 installer, a NodeJS 16 installer, and a Docker installer — each set to install automatically the first time a pipeline needs them.

The screenshot shows the Jenkins web interface at the URL `52.66.237.177:8080/manage/configureTools/`. The page title is "Jenkins / Manage Jenkins ▾ / Tools". The "JDK installations" section contains a list of tool configurations. The first configuration is named "jdk17" and has the "Install automatically" checkbox checked. Under this configuration, the "Install from adoptium.net" checkbox is also checked, and the "Version" is set to "jdk-17.0.8.1+1". There are "Add JDK" and "Add Installer" buttons. The "Git installations" section shows a single configuration named "Git". At the bottom of the page are "Save" and "Apply" buttons.

JDK installations

+ Add JDK

JDK

Name

jdk17

☒ Install automatically ?

Install from adoptium.net ?

Version ?

jdk-17.0.8.1+1

+ Add Installer

+ Add JDK

Git installations

Git

Save Apply

JDK tool configuration — auto-installed from Adoptium (jdk-17.0.8.1+1).

 **Jenkins**

Manage Jenkins

Tools



NodeJS installations

+

Add NodeJS

⋮

NodeJS

✕

Name

node16

✓

Install automatically

?

⋮

Install from nodejs.org

✕

Version

NodeJS 25.2.1

▼

For the underlying architecture, if available, force the installation of the 32bit package. Otherwise the build will fail

Force 32bit architecture

Global npm packages to install

Specify list of packages to install globally -- see npm install -g. Note that you can fix the packages version by using the syntax `packageName@version`

Global npm packages refresh hours

Duration, in hours, before 2 npm cache update. Note that 0 will always update npm cache

72

Save

Apply

NodeJS tool configuration — auto-installed from nodejs.org.

Akanksha Singh

linkedin.com/in/akanksha-singh-2780a6253

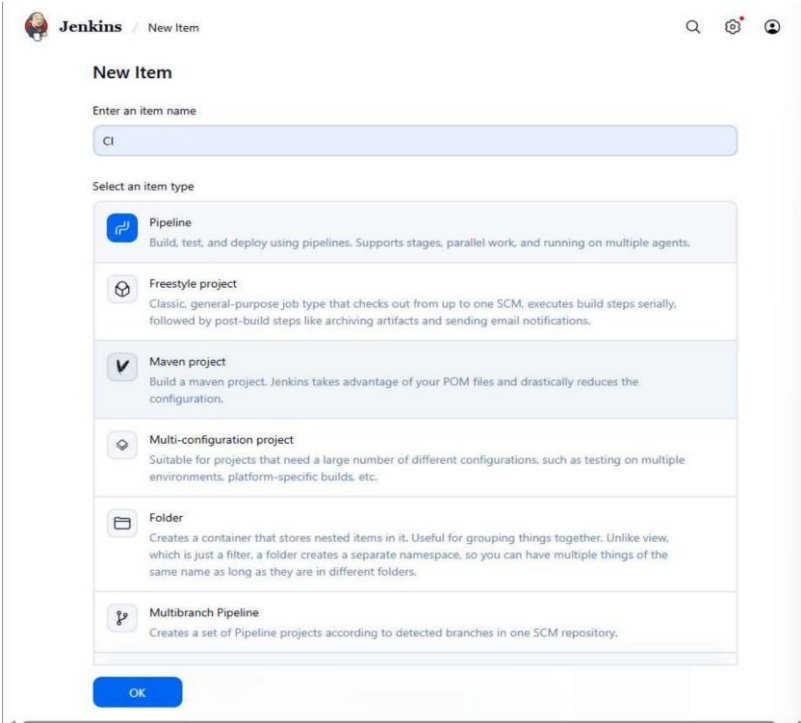
Page 15

The screenshot shows the Jenkins 'Tools' configuration page for Docker. At the top, there's a header with the Jenkins logo, 'Manage Jenkins' dropdown, and 'Tools' label. Below this, there are buttons for '+ Add Installer', '+ Add NodeJS', and '+ Add Docker'. The 'Docker installations' section contains a configuration card for 'Docker'. This card has a 'Name' field with 'docker', a checked 'Install automatically' checkbox, and a sub-section 'Download from docker.com' with a 'Docker version' field set to 'latest'. There are 'Add Installer' and 'Save' buttons at the bottom of the configuration card. The 'Save' button is highlighted in blue.

Docker tool configuration — latest version, auto-installed from docker.com.

8 Creating the CI Pipeline Job

A new Jenkins **Pipeline** job named CI was created. Rather than writing the pipeline logic directly into Jenkins' UI, the job was configured to pull a Jenkinsfile straight from the GitHub source repository (*Pipeline script from SCM*). This is the recommended approach in real projects: the pipeline definition lives as version-controlled code next to the application, so every change is tracked in Git history instead of being an invisible, unaudited edit inside Jenkins' web UI.



Creating a new Jenkins item of type 'Pipeline'.

 **Jenkins**

CI / Configure

Configure

General

Triggers

Pipeline

Advanced

Definition

Pipeline script from SCM

SCM ?
Git

Repositories ?

Repository URL ?
https://github.com/Akku690/Starbucks-Application.git

Credentials ?
- none - + Add

Advanced

+ Add Repository

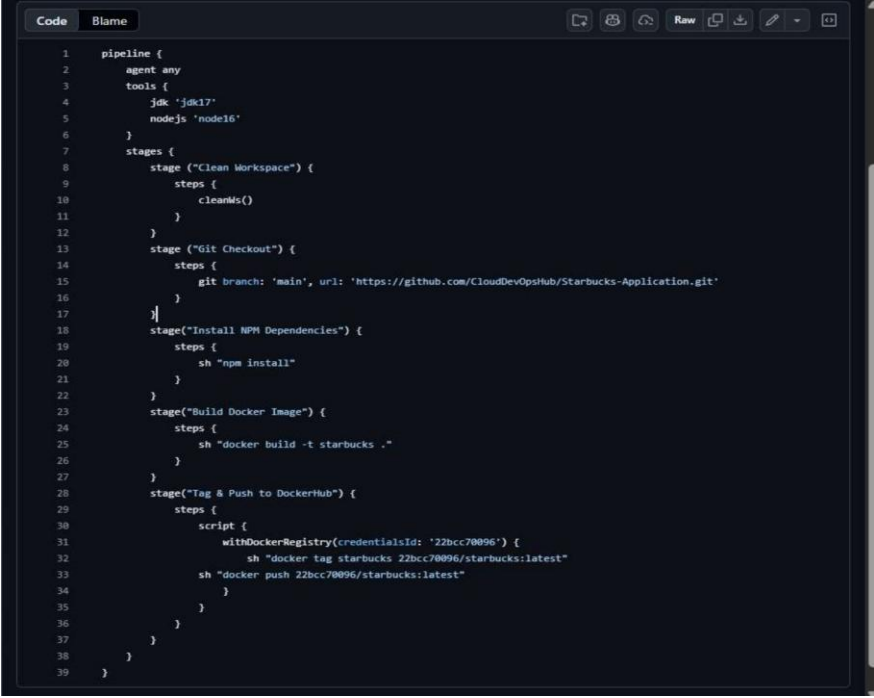
Branches to build ?

Branch Specifier (blank for 'any') ?
*/main

Save Apply

Pointing the pipeline at the GitHub repository and the 'main' branch, using 'Pipeline script from SCM'.

The CI Jenkinsfile



```

1  pipeline {
2      agent any
3      tools {
4          jdk 'jdk17'
5          nodejs 'node16'
6      }
7      stages {
8          stage ("Clean Workspace") {
9              steps {
10                  cleanWs()
11              }
12          }
13          stage ("Git Checkout") {
14              steps {
15                  git branch: 'main', url: 'https://github.com/CloudDevOpsHub/Starbucks-Application.git'
16              }
17          }
18          stage("Install NPM Dependencies") {
19              steps {
20                  sh "npm install"
21              }
22          }
23          stage("Build Docker Image") {
24              steps {
25                  sh "docker build -t starbucks ."
26              }
27          }
28          stage("Tag & Push to DockerHub") {
29              steps {
30                  script {
31                      withDockerRegistry(credentialsId: '22bcc70096') {
32                          sh "docker tag starbucks 22bcc70096/starbucks:latest"
33                          sh "docker push 22bcc70096/starbucks:latest"
34                      }
35                  }
36              }
37          }
38      }
39  }

```

The Jenkinsfile used for the CI stage — checkout, dependency install, Docker build, and push to Docker Hub.

Command	Why we use it
<code>tools { jdk 'jdk17'; nodejs 'node16' }</code>	Tells Jenkins which pre-configured tool installations (from Section 7) to inject into this pipeline's environment.
<code>stage('Clean Workspace') { cleanWs() }</code>	Wipes any leftover files from a previous build so every run starts from a completely clean state.
<code>stage('Git Checkout') { git branch: 'main', url: '...' }</code>	Clones the latest source code from the GitHub repository.
<code>stage('Install NPM Dependencies') { sh 'npm install' }</code>	Installs the application's JavaScript dependencies as defined in package.json.
<code>stage('Build Docker Image') { sh 'docker build -t starbucks .' }</code>	Packages the application and its dependencies into a Docker image using the project's Dockerfile.
<code>withDockerRegistry(credentialsId: '...') { sh 'docker push ...' }</code>	Authenticates to Docker Hub using a securely stored Jenkins credential, tags the image, and pushes it so it can later be pulled and run anywhere.

9 Troubleshooting: Real Pipeline Failures & Fixes

The first pipeline run did not succeed on the first try — and that's normal. Below are the actual errors encountered, exactly as they appeared in the Jenkins console output, along with the diagnosis and fix applied for each.

Jenkins / CI

Status

Changes

Build Now

Configure

Delete Pipeline

Full Stage View

Stages

Rename

Pipeline Syntax

Builds >

Filter

Today

#1 2:13 PM

CI

Stage View

Average stage times:

Declarative: Checkout SCM	Declarative: Tool Install	Clean Workspace	Git Checkout	Install NPM Dependencies
1s	26s	438ms	1s	756ms

#1 Jun 30 19:43 No Changes

1s 26s 438ms 1s 756ms fail

Permalinks

- Last build (#1), 15 min ago
- Last failed build (#1), 15 min ago
- Last unsuccessful build (#1), 15 min ago
- Last completed build (#1), 15 min ago

The CI job's first run failing at the 'Install NPM Dependencies' stage.

■ Error 1 — npm install fails with a missing shared library

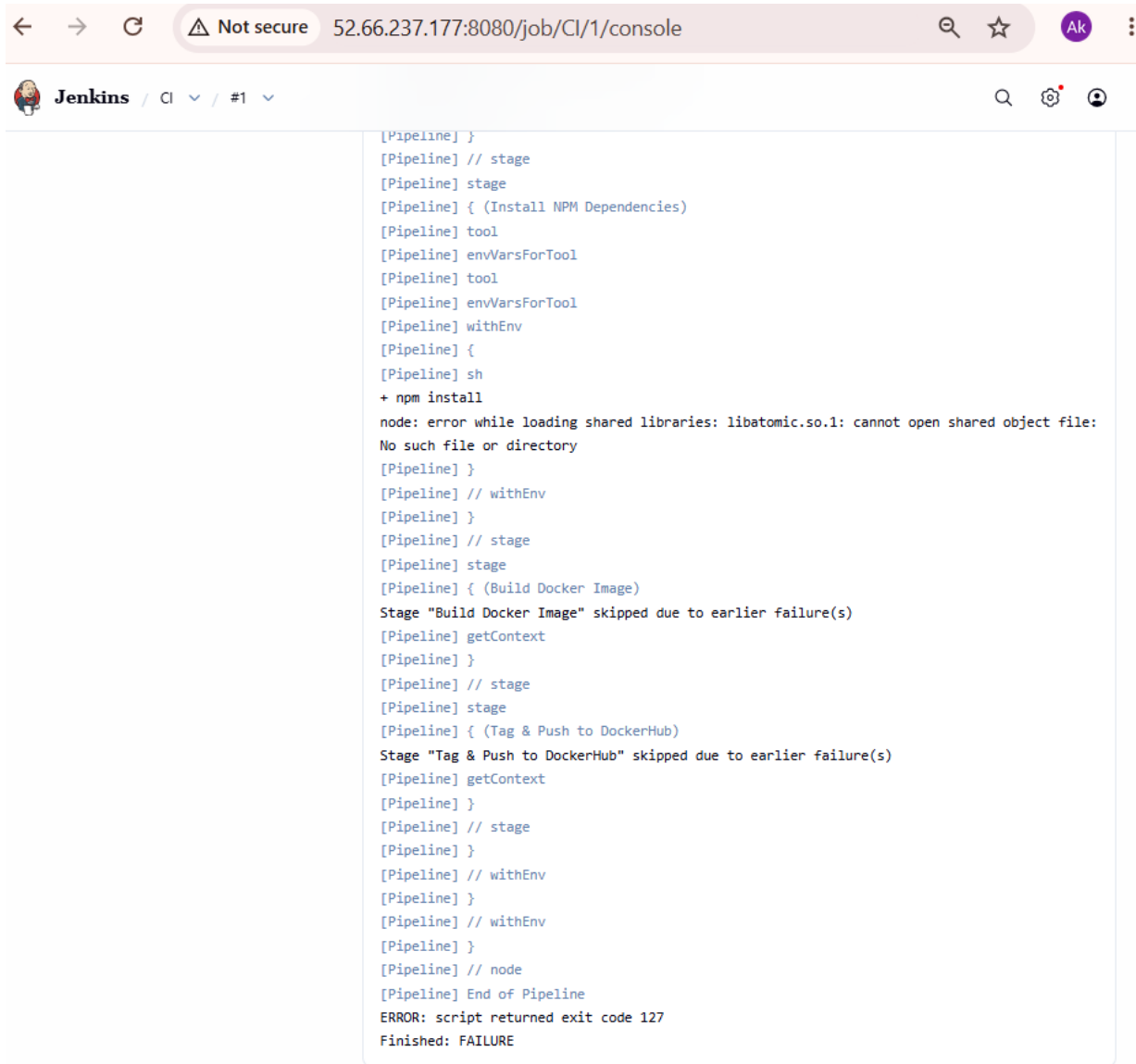
```
+ npm install
node: error while loading shared libraries: libatomic.so.1:
cannot open shared object file: No such file or directory
```

Root cause & Fix

The Node.js binary that Jenkins auto-installed depends on a low-level system library called `libatomic`, which was not present on this minimal Ubuntu server image. Because the very next stages (Docker build, tag & push) depend on this one succeeding, they were automatically **skipped** as a downstream consequence.

Fix — install the missing library directly on the Jenkins agent:

```
sudo apt-get update
sudo apt-get install -y libatomic1
```



```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Install NPM Dependencies)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] sh
+ npm install
node: error while loading shared libraries: libatomic.so.1: cannot open shared object file:
No such file or directory
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Docker Image)
Stage "Build Docker Image" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Tag & Push to DockerHub)
Stage "Tag & Push to DockerHub" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 127
Finished: FAILURE
```

Console output pinpointing the exact failing command and error message.

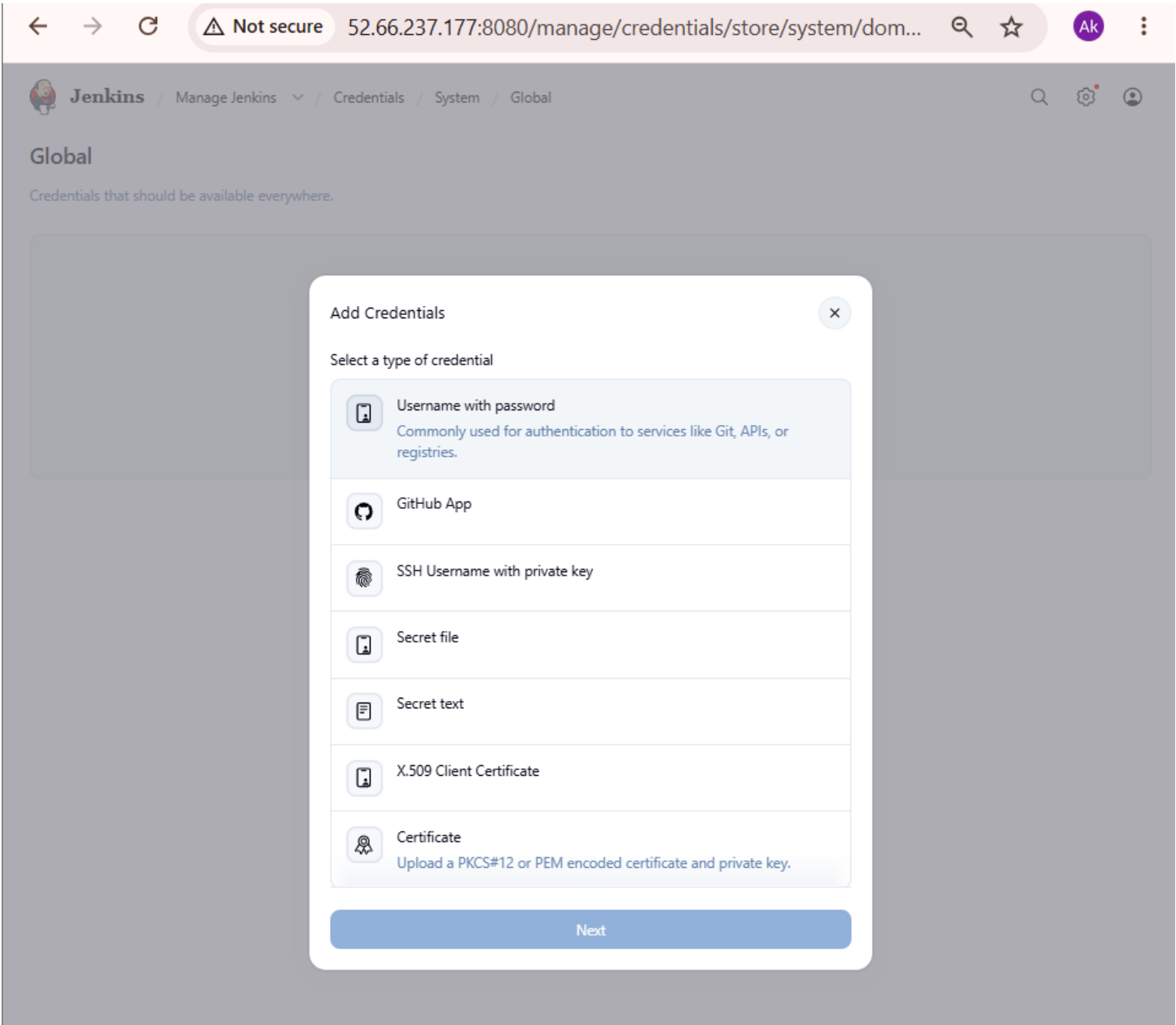
■ Error 2 — Docker Hub login fails with 'unauthorized'

```
$ docker login -u 22bcc70096 -p *****  
Error response from daemon: Get "https://registry-1.docker.io/v2/":  
unauthorized: incorrect username or password  
ERROR: docker login failed
```

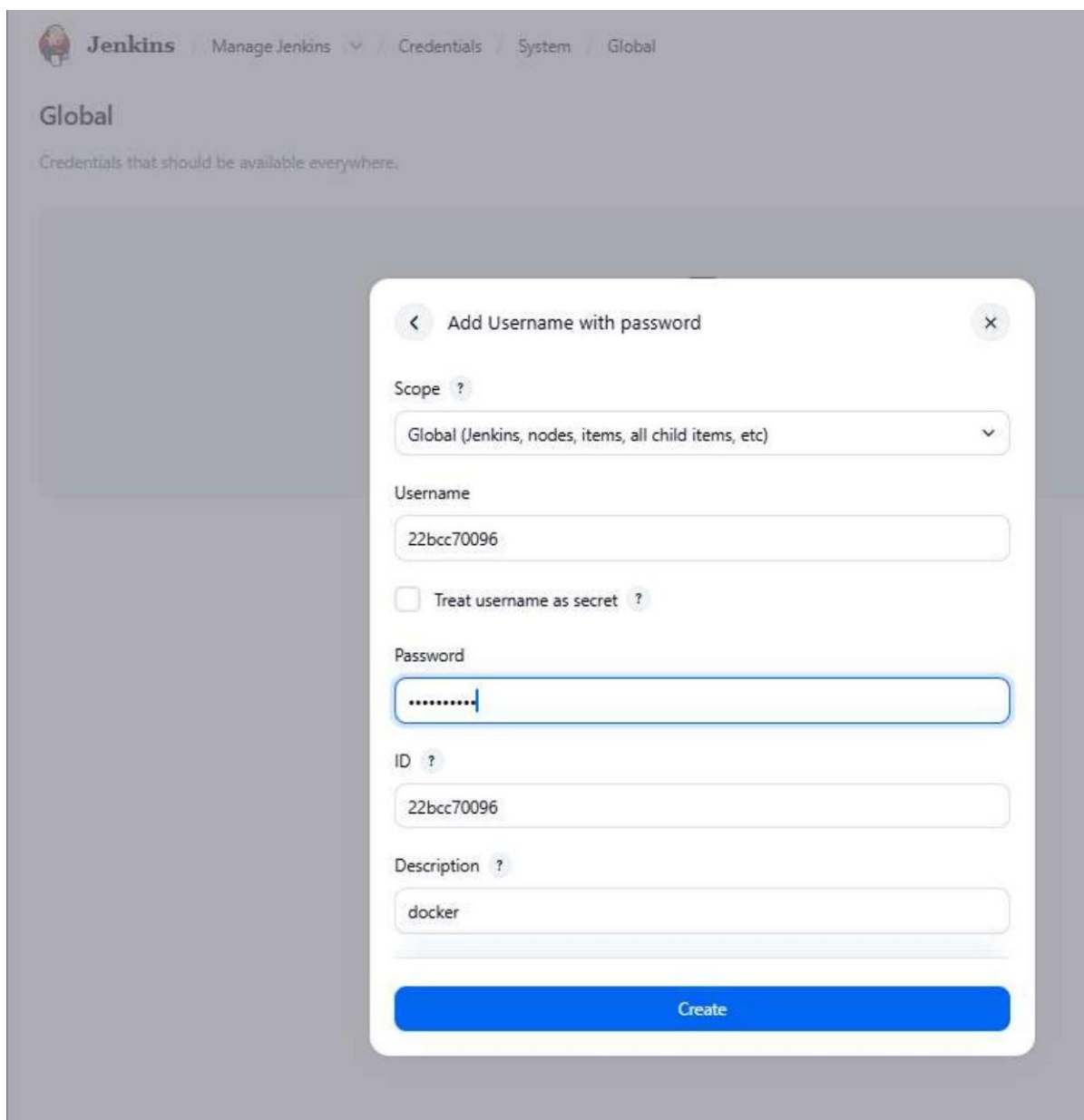
Root cause & Fix

Two separate issues surfaced here. First, the 'Tag & Push to DockerHub' stage referenced a Jenkins credential ID that hadn't actually been created yet in *Manage Jenkins* → *Credentials*. Second, once the credential was added, Docker Hub still rejected the account's regular login password — Docker Hub requires a dedicated **Personal Access Token** for CLI/API logins rather than the normal web password.

Fix, part A — create a Jenkins credential with an ID matching the one referenced in the Jenkinsfile's `withDockerRegistry(credentialsId: ...)` block:



Manage Jenkins > Credentials > Global > Add Credentials, selecting 'Username with password'.



Jenkins Manage Jenkins Credentials System Global

Global

Credentials that should be available everywhere.

< Add Username with password X

Scope ?
Global (Jenkins, nodes, items, all child items, etc) v

Username
22bcc70096

☐ Treat username as secret ?

Password
.....

ID ?
22bcc70096

Description ?
docker

Create

Docker Hub username entered as both the Username and the Credential ID (matching the Jenkinsfile reference), with a description for clarity.

Fix, part B — generate a Docker Hub Access Token instead of using the account password:

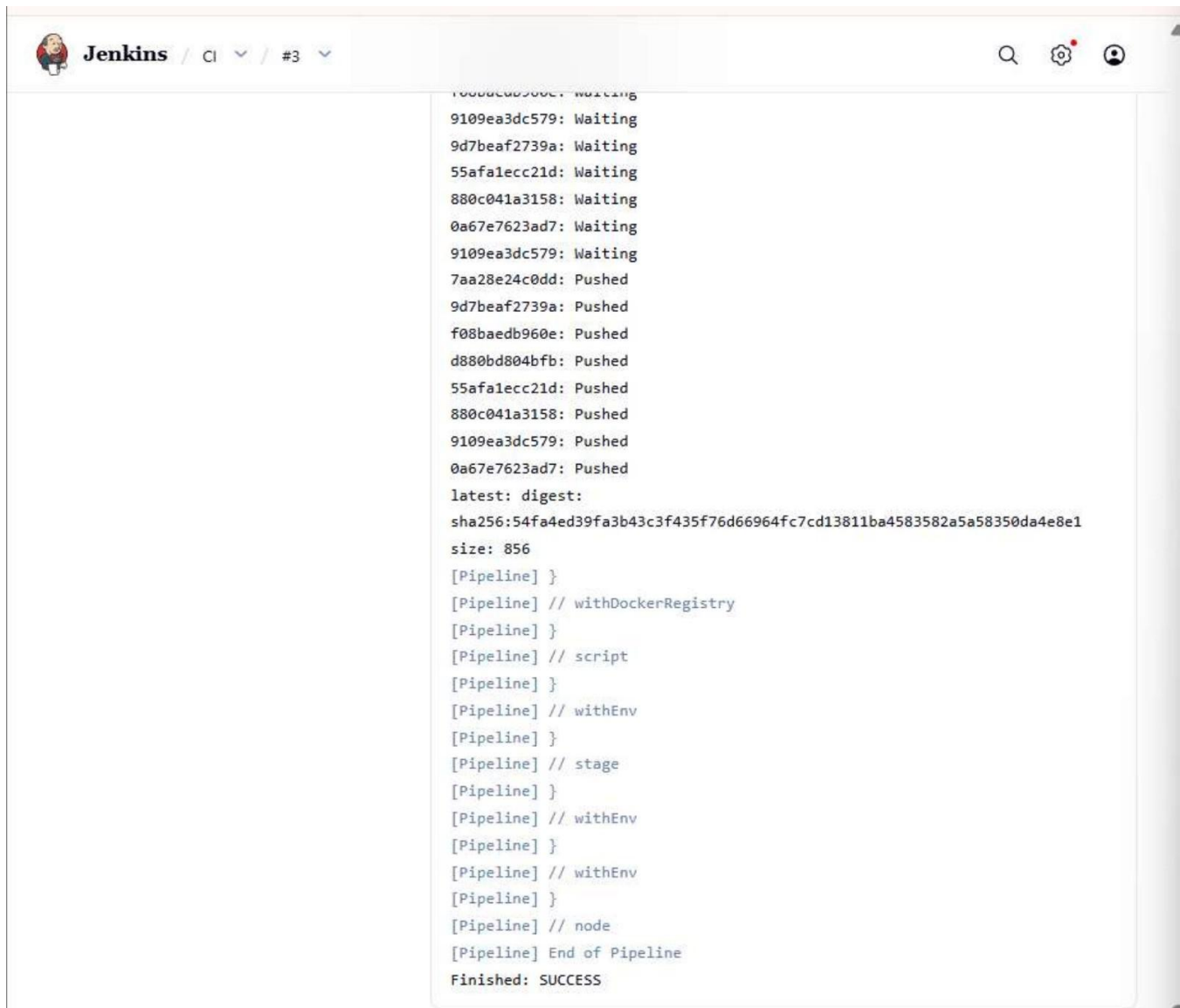
Fix — generate and use a Docker Hub Access Token:

1. Go to **hub.docker.com** → click your **profile icon** (top right) → **Account Settings**.
2. Go to **Security** → **Personal access tokens** (or **New Access Token**).
3. Click **Generate New Token**, give it a description like **jenkins-ci**, and set permissions to **Read & Write** (needed for docker push).
4. Copy the generated token immediately (it's shown only once).
5. In **Jenkins**, go to **Manage Jenkins** → **Credentials** → **Global** → **22bcc70096** (the credential in image 2), click the **pencil (Edit)** icon.
6. Keep the **Username** as **22bcc70096**.
7. Replace the **Password** field with the new **Access Token** (not your Docker Hub account login password).
8. Save, then **re-run the pipeline**.

Steps to generate a Docker Hub Personal Access Token with Read & Write permissions.

For security, the generated token itself is intentionally not shown in this document — access tokens should always be treated as secrets and never published, even in a walkthrough.

The token was pasted into the **Password** field of the existing Jenkins credential (replacing the rejected account password), keeping the Username and Credential ID unchanged. After saving and re-running the pipeline, the build completed successfully end-to-end.



The image shows a Jenkins console output for a CI/CD pipeline. The top bar indicates the pipeline is named 'CI' and is at step '#3'. The output log shows a series of Docker image builds and pushes. The first build is 'Waiting', followed by several 'Pushed' status messages for different image IDs. The final output shows the 'latest' digest and a 'Finished: SUCCESS' message.

```
9109ea3dc579: Waiting
9d7beaf2739a: Waiting
55afa1ecc21d: Waiting
880c041a3158: Waiting
0a67e7623ad7: Waiting
9109ea3dc579: Waiting
7aa28e24c0dd: Pushed
9d7beaf2739a: Pushed
f08baedb960e: Pushed
d880bd804bfb: Pushed
55afa1ecc21d: Pushed
880c041a3158: Pushed
9109ea3dc579: Pushed
0a67e7623ad7: Pushed
latest: digest:
sha256:54fa4ed39fa3b43c3f435f76d66964fc7cd13811ba4583582a5a58350da4e8e1
size: 856
[Pipeline] }
[Pipeline] // withDockerRegistry
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Successful re-run: the image is built, tagged, and pushed to Docker Hub — 'Finished: SUCCESS'.

 Jenkins

CI / #3

Search

Settings

Profile

Status

Changes

Console Output

Edit Build Information

Delete build '#3'

Git Build Data

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

✓ #3 (Jun 30, 2026, 2:38:28 PM)

Add descriptionKeep this build forever

Started by user admin

Started 2 min 0 sec agoTook 1 min 15 sec

git

Revision: 1dd3131a8ab0f03dad85a91340d8cf3e4a1a3f24

Repository: https://github.com/Akku690/Starbucks-Application.git

refs/remotes/origin/main

git

Revision: c5379bc644c692ccff858de6da5cfc8ff74850f8

Repository: https://github.com/CloudDevOpsHub/Starbucks-Application.git

refs/remotes/origin/main

</>

No changes.

hub

Search

Notifications

More

Profile

Repositories

All repositories within the 22bcc70096 namespace.

Search by repository name

All content

Create a repository

Name	Last Pushed ↑	Contains	Visibility	Scout
22bcc70096/starbucks	2 minutes ago	IMAGE	Public	Inactive
22bcc70096/akankshasingh	16 days ago		Public	Inactive
22bcc70096/testapp	about 1 month ago		Public	Inactive

1-3 of 3

The 'starbucks' image now visible in the Docker Hub repository, pushed a couple of minutes earlier.

Akanksha Singh

linkedin.com/in/akanksha-singh-2780a6253

Page 28

10 Building the CD (Continuous Deployment) Pipeline

With a working image published to Docker Hub, a second Jenkins pipeline job, CD, was created to handle deployment — pulling the freshly built image and (re)starting it as a running container on the server.

CD with Another Approach

```
pipeline {
  agent any
  tools {
    jdk 'jdk17'
    nodejs 'node16'
  }

  Stage("Deploy to Container"){
    steps{
      sh "docker run -d --name starbucks -p 3000:3000 22bcc70096/starbucks:latest"
    }
  }
}
```

Here's a proper CD (Continuous Deployment) Jenkins Pipeline for your Starbucks Docker application:

```
pipeline {
  agent any

  tools {
    jdk 'jdk17'
    nodejs 'node16'
  }

  stages {
    stage("Deploy to Docker Container") {
      steps {
        sh """
          # Stop existing container if it exists
          docker stop starbucks || true

          # Remove existing container
          docker rm starbucks || true

          # Pull latest image
          docker pull 22bcc70096/starbucks:latest

          # Run new container
          docker run -d \
            --name starbucks \
            -p 3000:3000 \
            22bcc70096/starbucks:latest
        """
      }
    }
  }

  post {
    success {
      echo 'Application deployed successfully!'
    }
    failure {
      echo 'Deployment failed!'
    }
  }
}
```

 Jenkins / All ▾ / New Item



New Item

Enter an item name

CD

Select an item type

 Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

 Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.

 Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

 Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

 Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.

 Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.

OK

Creating a second Pipeline job named 'CD', dedicated to deployment.

The screenshot shows the Jenkins web interface for configuring a pipeline. The browser address bar indicates the URL is `52.66.237.177:8080/job/CD/configure`. The Jenkins logo and 'CD / Configure' breadcrumb are at the top. On the left, a sidebar lists configuration options: General, Triggers, Pipeline (selected), and Advanced. The main area is titled 'Configure' and 'Pipeline'. It includes a description: 'Define your Pipeline using Groovy directly or pull it from source control.' Below this, a 'Definition' dropdown is set to 'Pipeline script'. A 'Script' section contains a Groovy pipeline script with line numbers 1 through 15. The script defines a pipeline with an 'any' agent, tools for 'jdk' and 'nodejs', and a stage named 'Deploy to Docker Container' with a shell step to stop an existing container. A checkbox for 'Use Groovy Sandbox' is checked. A 'Pipeline Syntax' link is below the script. At the bottom, an 'Advanced' section has a dropdown menu. 'Save' and 'Apply' buttons are at the very bottom.

← → ↻ ⚠ Not secure 52.66.237.177:8080/job/CD/configure 🔍 ☆ Ak ⋮

Jenkins / CD / Configure 🔍 ⚙️ 👤

Configure

- ⚙️ General
- 🕒 Triggers
- 📜 Pipeline**
- 🔧 Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script ▾

Script ?

```
1 pipeline {
2   agent any
3
4   tools {
5     jdk 'jdk17'
6     nodejs 'node16'
7   }
8
9   stages {
10
11     stage('Deploy to Docker Container') {
12       steps {
13         sh '''
14           # Stop existing container if it exists
15           docker stop starbucks || true
16         '''
17       }
18     }
19   }
20 }
```

try sample Pipeline... ▾

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

Advanced

Advanced ▾

Save Apply

The CD Jenkinsfile: stop and remove any existing container, pull the latest image, and run a fresh container.

Command	Why we use it
<code>docker stop starbucks true</code>	Stops the currently running container if one exists. The ' true' prevents the pipeline from failing on the very first deployment, when no container exists yet to stop.
<code>docker rm starbucks true</code>	Removes the stopped container so a new one with the same name can be created without a naming conflict.
<code>docker pull 22bcc70096/starbucks:latest</code>	Downloads the newest image version that CI just pushed to Docker Hub.
<code>docker run -d --name starbucks -p 3000:3000 22bcc70096/starbucks:latest</code>	Starts a new container in detached (background) mode, mapping port 3000 on the host server to port 3000 inside the container, where the application listens.

Jenkins / CD

CD

Stage View

Average stage times: (full run time: ~4s)

Stage	Tool Install	Deploy to Docker Container	Post Actions
Average	178ms	3s	86ms
Actual	178ms	3s	86ms

Permalinks

Builds >

Today

✓ #1 2:50 PM

First manual run of the CD job, completing successfully in around 4 seconds.

11 Troubleshooting: Application Not Reachable

Even though the container was confirmed running, opening `http://<public-ip>:3000` in a browser timed out.

You can see starbucks at 3000 port

You can see image in dockerhub it has been pushed .

Now copy the public ip paste it on 3000 port on browser.

Now you will be able to see starbucks .



This site can't be reached

52.66.237.177 took too long to respond.

Try:

- Checking the connection
- [Checking the proxy and the firewall](#)
- [Running Windows Network Diagnostics](#)

ERR_CONNECTION_TIMED_OUT

Reload

Details

Browser error: 'This site can't be reached / ERR_CONNECTION_TIMED_OUT'.

■ Error 3 — site unreachable on port 3000

Root cause & Fix

The container itself was healthy — the problem was one layer up, at the network boundary. AWS Security Groups block all inbound traffic by default, and while port 3000 was used inside the `docker run` command, it had never been opened on the EC2 instance's firewall the way ports 22, 80, 443 and 8080 had been in Section 2.

Fix — add an inbound rule for port 3000 on the EC2 Security Group:

- Type: Custom TCP
- Port range: 3000
- Source: 0.0.0.0/0 (or restricted to a known IP for better security)

Edit inbound rules Info

Inbound rules control the incoming traffic that's allowed to reach the instance.

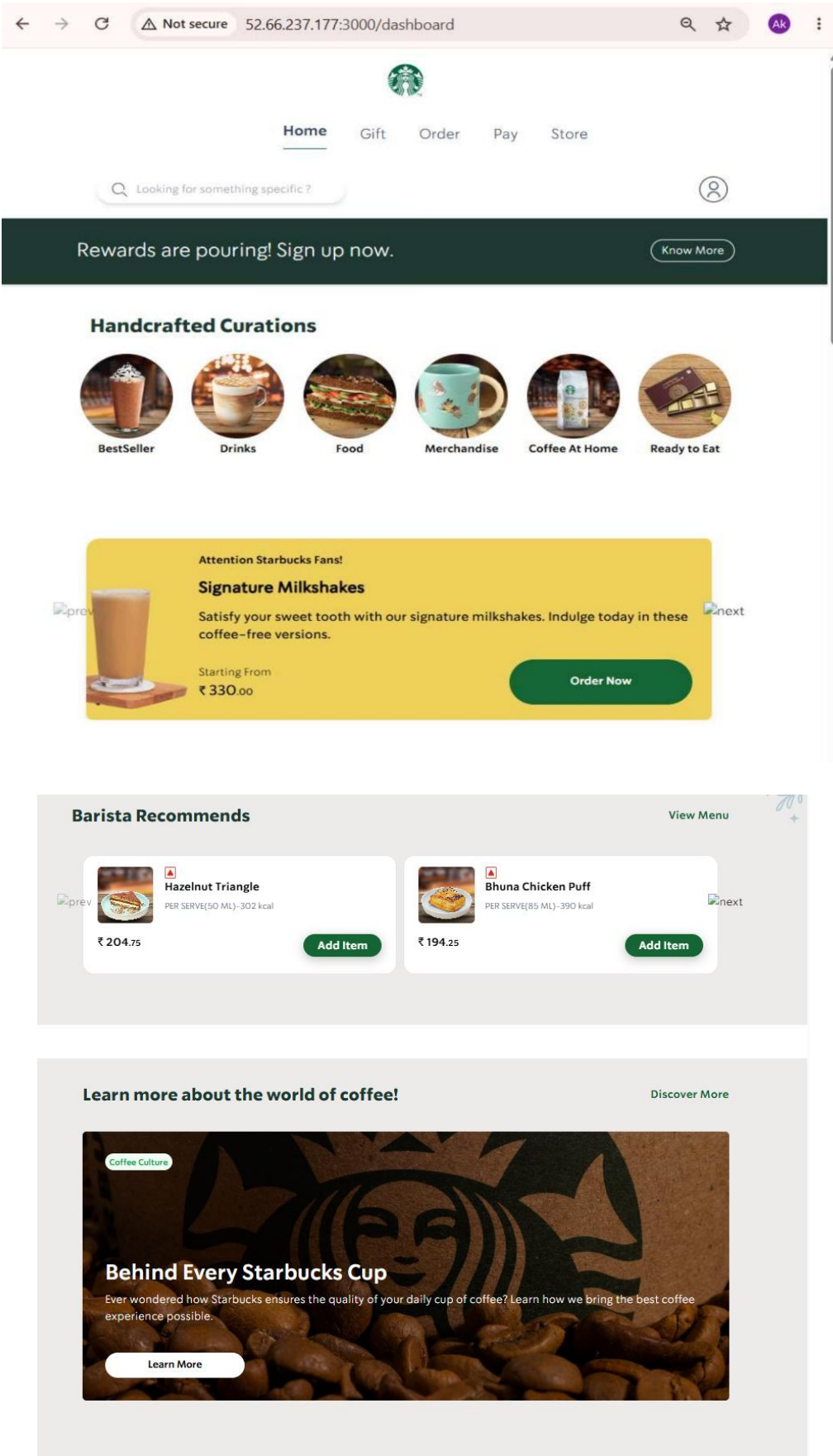
Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info		
sgr-0ce174adfb9e596b8	HTTPS	TCP	443	C...	Q	0.0.0.0/0 X	Delete
sgr-0fac35925ce618f84	HTTP	TCP	80	C...	Q	0.0.0.0/0 X	Delete
sgr-0886ad3deaa13b783	SSH	TCP	22	C...	Q	0.0.0.0/0 X	Delete
sgr-0e0497c6903baf88	Custom TCP	TCP	8080	C...	Q	0.0.0.0/0 X	Delete
-	Custom TCP	TCP	3000	A...	Q 0.0.0.0/0	0.0.0.0/0 X	Delete

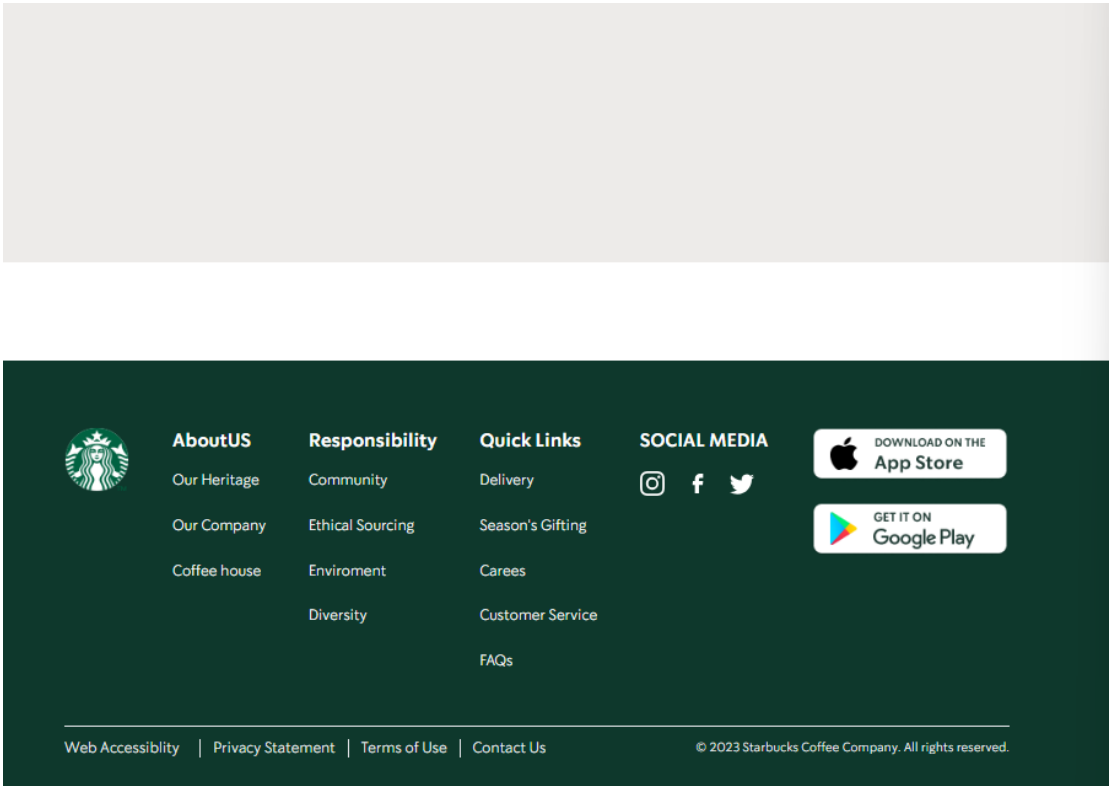
[Add rule](#)

⚠ Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

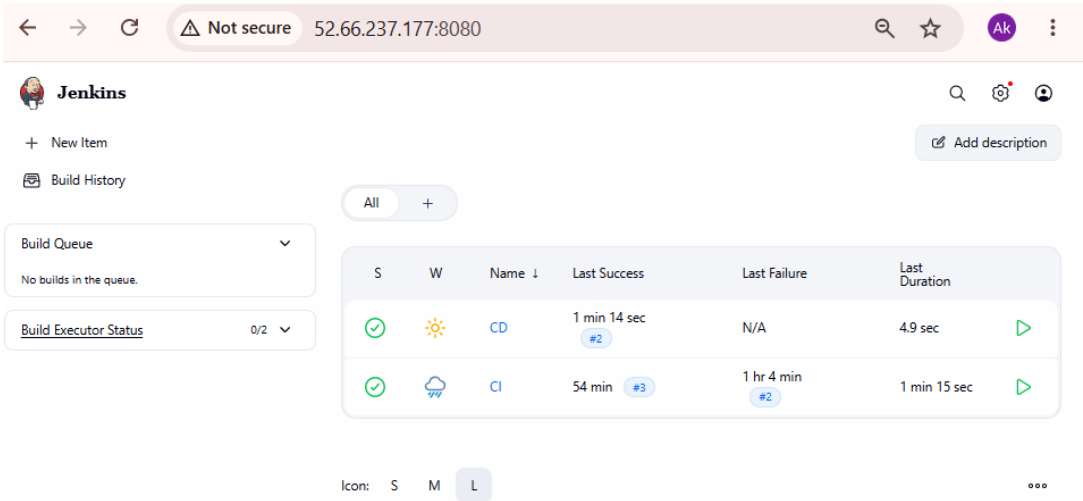
[Cancel](#) [Preview changes](#) [Save rules](#)

Adding the missing inbound rule for port 3000 to the EC2 Security Group.





The application, now successfully reachable and rendering in the browser on port 3000.



12 Chaining CI → CD Automatically

Up to this point, the CD job had to be triggered manually after every successful CI run. The final step was to remove that manual hand-off entirely: the CD job was configured with a build trigger so that it fires automatically the moment the CI job finishes with a stable (successful) result.

The screenshot shows the Jenkins web interface for configuring a job named 'CD'. The browser address bar indicates the URL is 52.66.237.177:8080/job/CD/configure. The left sidebar shows the 'Configure' page with tabs for General, Triggers, Pipeline, and Advanced. The 'Triggers' tab is selected. Under the 'Triggers' section, the option 'Build after other projects are built' is checked. Below this, 'Projects to watch' is set to 'CI'. The trigger condition is set to 'Trigger only if build is stable'. Other trigger options like 'Build periodically', 'GitHub hook trigger for GITScm polling', 'Poll SCM', and 'Trigger builds remotely' are unchecked. The 'Pipeline' section shows the 'Definition' set to 'Pipeline script'. The script content is visible in a text area, showing a Groovy pipeline definition with agent, tools, and stages sections.

Configure

- General
- Triggers
- Pipeline
- Advanced

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☒ Build after other projects are built ?

Projects to watch

CI

☒ Trigger only if build is stable

☐ Trigger even if the build is unstable

☐ Trigger even if the build fails

☐ Always trigger, even if the build is aborted

☐ Build periodically ?

☐ GitHub hook trigger for GITScm polling ?

☐ Poll SCM ?

☐ Trigger builds remotely (e.g., from scripts) ?

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   tools {
5     jdk 'jdk17'
6     nodejs 'node16'
7   }
8
9   stages {
10
```

Save Apply

CD job configuration: 'Build after other projects are built', watching the CI project, triggered only if the build is stable.

With this trigger in place, running the CI job end-to-end also kicks off the CD job automatically — verified below by the CD console output, which shows it was started by the upstream CI project rather than by a person.

Jenkins / CI

Status

</> Changes

Build Now

Configure

Delete Pipeline

Full Stage View

Stages

Rename

Pipeline Syntax

Add description

Stage View

Average stage times:
(full run time: ~1min 15s)

	Declarative: Checkout SCM	Declarative: Tool Install	Clean Workspace	Git Checkout	Install NPM Dependencies
#4 Jul 01 01:54 No Changes	1s	200ms	363ms	1s	13s
#3 Jun 30 20:08 No Changes	1s	182ms	368ms	1s	12s
#2 Jun 30 19:59 No Changes	1s	197ms	366ms	1s	15s
#1 Jun 30 19:43 No Changes	1s	26s	438ms	1s	756ms fail

Permalinks

- Last build (#3), 5 hr 46 min ago
- Last stable build (#3), 5 hr 46 min ago
- Last successful build (#3), 5 hr 46 min ago
- Last failed build (#2), 5 hr 55 min ago
- Last unsuccessful build (#2), 5 hr 55 min ago
- Last completed build (#3), 5 hr 46 min ago

 Jenkins

CI

▼

#4

🔍

 1



Status

</> Changes

📄 Console Output

📄 Edit Build Information

🗑️ Delete build '#4'

🔑 Git Build Data

🔑 Git Build Data

🔗 Pipeline Overview

🔄 Restart from Stage

🔄 Replay

☰ Pipeline Steps

📁 Workspaces

⬅️ Previous Build

✅ #4 (Jun 30, 2026, 8:24:39 PM)

📝 Add description

🔒 Keep this build forever

 Started by user [admin](#)

🔑 git

Revision: 1dd3131a8ab0f03dad85a91340d8cf3e4a1a3f24
Repository: <https://github.com/Akku690/Starbucks-Application.git>

- refs/remotes/origin/main

🔑 git

Revision: c5379bc644c692ccff858de6da5cfc8ff74850f8
Repository: <https://github.com/CloudDevOpsHub/Starbucks-Application.git>

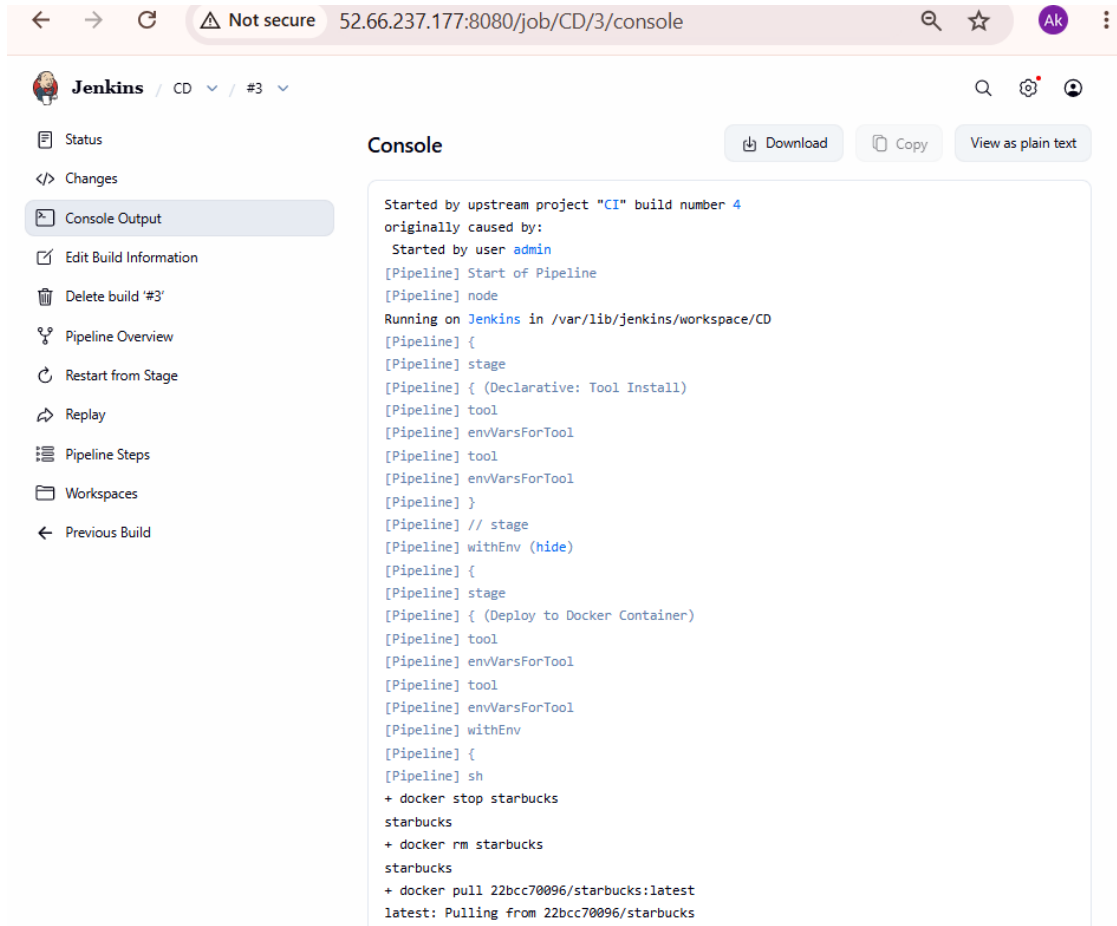
- refs/remotes/origin/main

</>

No changes.

Started 1 min 37 sec ago

Took [1 min 14 sec](#)



The screenshot shows the Jenkins web interface in a browser. The address bar indicates the URL is 52.66.237.177:8080/job/CD/3/console. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Edit Build Information, Delete build '#3', Pipeline Overview, Restart from Stage, Replay, Pipeline Steps, Workspaces, and Previous Build. The main console area displays the following output:

```
Started by upstream project "CI" build number 4
originally caused by:
  Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/CD
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Tool Install)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv (hide)
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Deploy to Docker Container)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] sh
+ docker stop starbucks
starbucks
+ docker rm starbucks
starbucks
+ docker pull 22bcc70096/starbucks:latest
latest: Pulling from 22bcc70096/starbucks
```

CD console output: 'Started by upstream project "CI" build number 4' — confirming full pipeline automation.

13 Key Takeaways

- A CI/CD pipeline is only as reliable as its infrastructure — sizing the EC2 instance correctly and opening the right Security Group ports upfront avoids a lot of downstream debugging.
- Keeping the pipeline definition as a Jenkinsfile in source control (rather than pasted into Jenkins' UI) makes every change to the build/deploy process auditable through Git.
- Almost every real pipeline fails at least once before it fully works — missing OS libraries, wrong credential IDs, and firewall gaps are some of the most common first-run issues, and the Jenkins console log is the fastest way to diagnose them.
- Docker Hub (and most registries) expect a Personal Access Token for automated logins, not an account password — and any generated token should be treated as a secret and never shared or committed to a repository.
- Automating the hand-off between CI and CD (build trigger on 'stable' status) is what turns two separate manual jobs into one true continuous delivery pipeline.

Thank you for reading! This project was built and documented end-to-end as a hands-on way to practice real-world DevOps — from provisioning cloud infrastructure to automating a full build-and-deploy pipeline with Jenkins and Docker.

Connect with me on LinkedIn: [linkedin.com/in/akanksha-singh-2780a6253](https://www.linkedin.com/in/akanksha-singh-2780a6253)

